Summary: Training a CNN on Stellar Spectra

What I did and how:

I implemented a standard Convolutional Neural Network (CNN) in PyTorch to predict three physical properties of stars, effective temperature (t_eff), surface gravity (log_g), and iron abundance (fe_h) from their spectra. The dataset contained 8914 stars, each with a spectrum of 16384 measurements, downloaded from HuggingFace (GALAH4 dataset).

The pipeline consisted of loading and normalizing the spectra using a log transformation, splitting the data into training (80%), validation (10%), and test (10%) sets, and feeding the data in batches of 32 using DataLoaders. The CNN architecture consisted of three convolutional blocks (each with Conv1d, ReLU, and MaxPool1d) followed by two fully connected layers, resulting in approximately 33 million trainable parameters. The model was trained for 20 epochs using the Adam optimizer with a learning rate of 0.001 and MSE loss.

What results I obtained:

The training and validation loss curves showed rapid learning in the first 5 epochs, after which overfitting began, training loss continued decreasing while validation loss plateaued around 4000-6000. On the test set, the model achieved MSE of 8948 for t_eff, 7.29 for log_g, and 5.23 for fe_h. The predicted vs true plots revealed that the model predicted t_eff reasonably well, but struggled significantly with log_g and fe_h, often predicting near-constant values regardless of the true label.

Challenges encountered and what could be improved:

The main challenge was overfitting, which appeared early in training. Several improvements could address this; adding dropout layers to regularize the model, using early stopping to halt training at the best validation epoch, normalizing the labels before training (the three labels have very different scales which may have caused the model to prioritize t_eff), training for more epochs with a lower learning rate, or using a more efficient architecture with fewer parameters to reduce overfitting.